Ferramentas de apoio à Reengenharia de Software

Integrantes: Gabriel Galhardone Felipe, João Paulo Candido, Miguel Costacurta

RA:24021292-2, 24000797-2, 24011302-2

1. Introdução

A reengenharia de software consiste em compreender e transformar um sistema já existente para melhorar sua manutenção, sua estrutura interna e sua capacidade de evolução. Em muitos casos, ela é aplicada a sistemas legados que continuam sendo importantes para o negócio, mas já apresentam sinais de desgaste técnico. Em vez de simplesmente substituir tudo, a reengenharia procura preservar o valor construído ao longo do tempo e reduzir riscos de mudanças bruscas.

Sistemas legados costumam exigir esse tipo de intervenção quando acumulam documentação fraca, alto acoplamento, dependência de poucos especialistas, dificuldade para corrigir defeitos e grande chance de efeitos colaterais em mudanças pequenas. Nessas condições, o custo de manutenção cresce e o time perde previsibilidade. Ferramentas de análise ajudam justamente a recuperar conhecimento técnico e a transformar percepções subjetivas em evidências.

Neste relatório foram escolhidas três ferramentas com papéis complementares: SonarQube, Checkstyle e VisualVM. As duas primeiras contribuem principalmente com análise estática, enquanto a terceira acrescenta observação dinâmica do comportamento do sistema. Juntas, elas apoiam um processo inicial de reengenharia orientado por métricas, padrões e sinais reais de execução.

2. Fundamentação conceitual

Engenharia reversa e reengenharia não são sinônimos. A engenharia reversa tem como foco compreender o sistema existente: sua estrutura, dependências, organização e regras que estão embutidas no código. Já a reengenharia utiliza esse conhecimento para promover melhorias planejadas, como refatoração, atualização de arquitetura, reorganização de módulos e produção de documentação mais confiável. Em resumo, a engenharia reversa descobre; a reengenharia transforma.

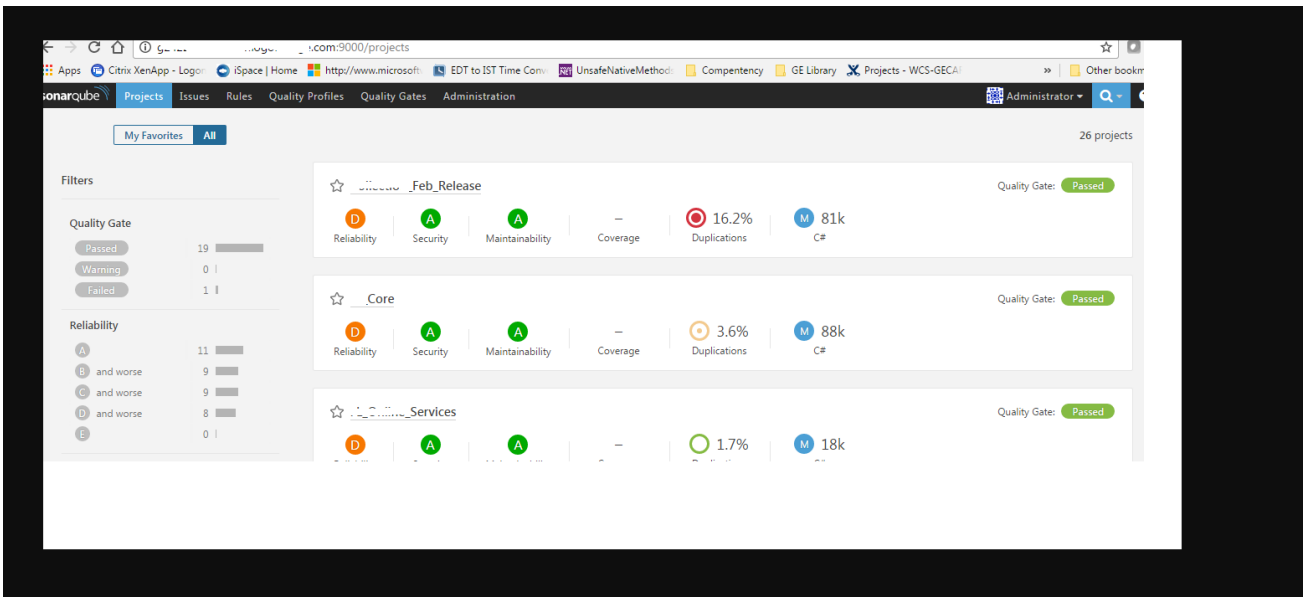
Também é necessário diferenciar análise estática e análise dinâmica. A análise estática examina o software sem executá-lo, permitindo identificar complexidade, duplicação, violações de padrão, dependências e métricas de manutenção. A análise dinâmica observa o sistema em funcionamento e torna visíveis aspectos como consumo de memória, uso de CPU, threads, gargalos e vazamentos. Na prática, a reengenharia se beneficia mais quando essas duas abordagens são combinadas.

3. Apresentação das ferramentas escolhidas

3.1 SonarQube

O SonarQube é uma plataforma de inspeção contínua de código voltada à análise estática. Seu objetivo principal é reunir, em um único painel, indicadores que ajudam a avaliar a saúde técnica do software. Entre as informações extraídas estão bugs potenciais, vulnerabilidades, code smells, duplicação, complexidade e cobertura de testes. Para a reengenharia, isso é valioso porque permite localizar módulos frágeis, definir prioridades e acompanhar a evolução da qualidade ao longo das melhorias.

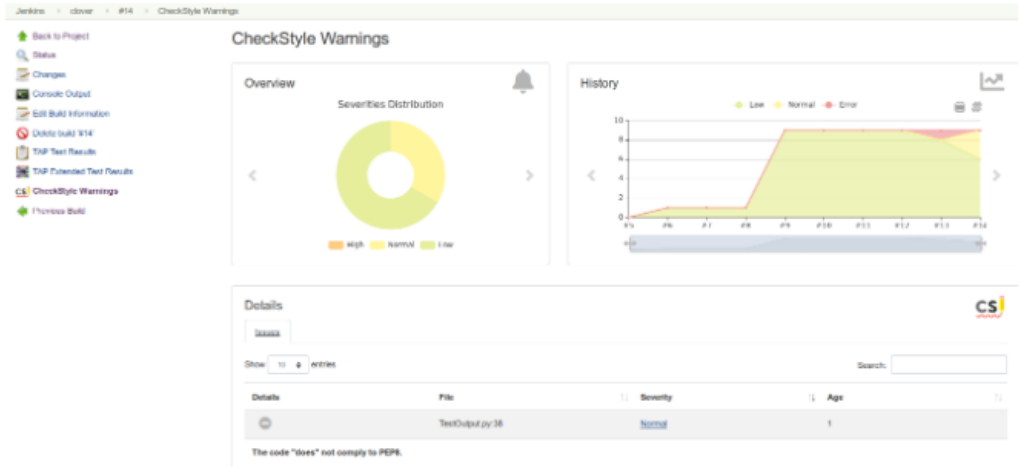
Como vantagens, destacam-se a boa visualização de resultados, a integração com pipelines e a possibilidade de trabalhar com critérios de qualidade. Sua principal limitação é que ele não mostra sozinho o comportamento do sistema em execução. Assim, é excelente para entender a estrutura do código, mas insuficiente para explicar certos problemas de desempenho ou concorrência.



3.2 Checkstyle

O Checkstyle é uma ferramenta de análise estática voltada principalmente a projetos Java. Seu foco é verificar conformidade com convenções de codificação, nomenclatura, organização de imports, tamanho de métodos e outras regras de estilo. Embora pareça mais simples que o SonarQube, ele tem utilidade real em reengenharia porque a falta de padronização aumenta o esforço de leitura e dificulta a manutenção coletiva do código.

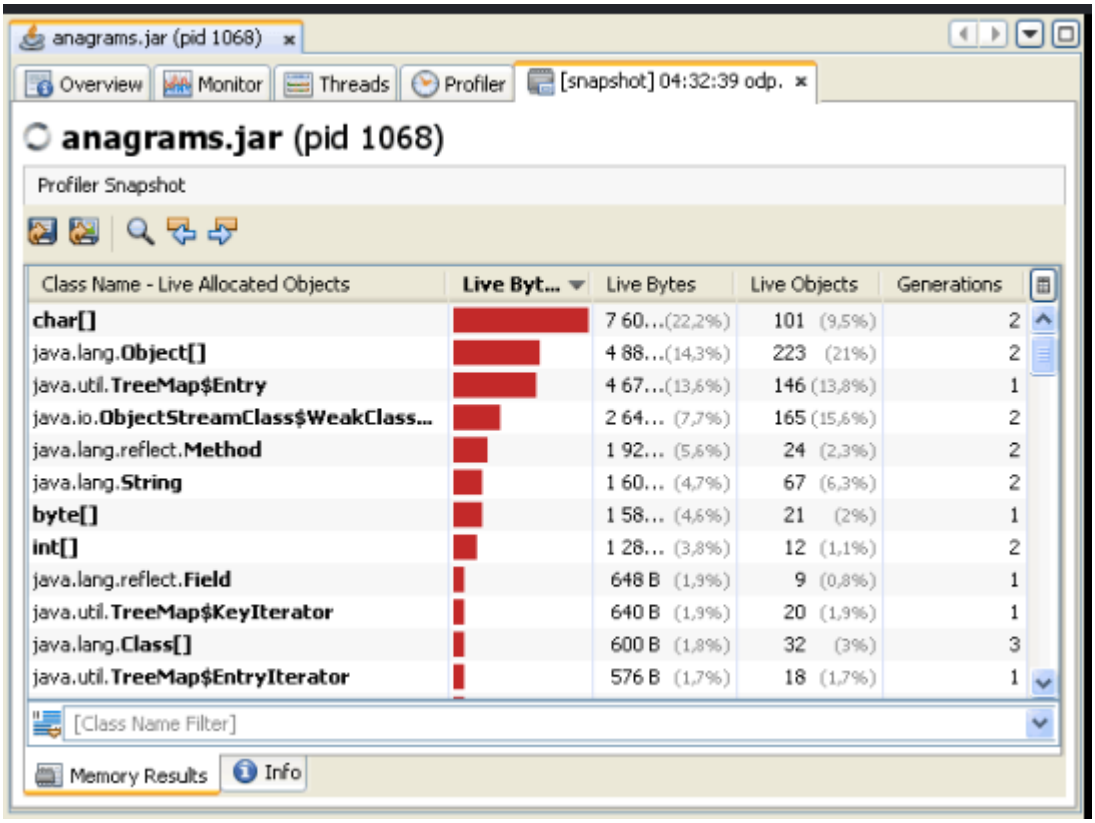
Os artefatos produzidos pelo Checkstyle ajudam a revelar inconsistências que prejudicam a compreensão do sistema. Ao padronizar a escrita do código, a equipe reduz ruído visual, melhora a legibilidade e cria uma base mais uniforme para futuras refatorações. Sua limitação está no foco mais restrito: ele não substitui análises arquiteturais mais amplas e tampouco mostra comportamento em tempo de execução.



3.3 VisualVM

O VisualVM trabalha de forma diferente, pois sua força está na análise dinâmica de aplicações Java. Ele monitora CPU, memória, threads, heap dumps e outros sinais de execução. Em sistemas legados, isso é importante quando a equipe precisa entender lentidão, travamentos, vazamentos de memória ou comportamento inconsistente que não aparece apenas pela leitura do código. Seu apoio à reengenharia está em validar hipóteses levantadas na análise estática e revelar gargalos reais. Uma classe

aparentemente simples pode se mostrar crítica quando observada sob carga ou em execução prolongada. Como vantagem, a ferramenta fornece diagnósticos úteis para desempenho e uso de recursos. Como limitação, depende de um ambiente de teste representativo para que os resultados sejam confiáveis.



4. Aplicação ao cenário proposto

O sistema apresentado no enunciado é um forte candidato à reengenharia porque reúne sintomas típicos de degradação: documentação ruim, dificuldade para pequenas alterações, dependência de poucas pessoas e defeitos em cascata. Isso mostra que o problema não está apenas em falhas isoladas, mas na perda gradual de compreensibilidade e controle sobre a base de código.

A ferramenta a ser usada primeiro seria o SonarQube. Antes de mudar o sistema, é necessário obter uma visão ampla dos arquivos mais problemáticos, da complexidade, das duplicações e dos sinais de baixa manutenibilidade. Com esse panorama, o grupo conseguiria priorizar onde investigar primeiro e evitar ações de manutenção baseadas apenas em intuição.

Nas etapas iniciais, seria importante extrair informações sobre módulos mais complexos, classes com maior incidência de problemas, regiões duplicadas e áreas sem testes suficientes. Em seguida, o Checkstyle ajudaria a localizar inconsistências de padronização que tornam o código mais difícil de ler. Por fim, o VisualVM entraria como complemento para observar se os módulos suspeitos também apresentam gargalos de CPU, memória ou concorrência.

A prioridade, portanto, seria uma combinação entre análise estática e dinâmica, começando pela estática. Essa sequência reduz custos e riscos porque organiza a investigação e torna o esforço de manutenção mais racional. Em vez de alterar partes do legado por tentativa e erro, a equipe passa a agir com base em métricas, evidências e hipóteses validadas.

5. Discussão

A pesquisa permite concluir que nenhuma ferramenta, sozinha, é suficiente para apoiar toda a reengenharia. O SonarQube ajuda mais na visão global da qualidade e na identificação de problemas de

manutenção; o Checkstyle ajuda mais na disciplina e na legibilidade do código; o VisualVM ajuda mais na compreensão do comportamento real do sistema em execução. Essa complementaridade mostra que a reengenharia é um processo técnico e analítico, não apenas uma troca de ferramenta.

Também se percebe que a análise estática, embora essencial, não resolve tudo. Ela não mostra diretamente gargalos de execução, bloqueios entre threads ou crescimento anormal de memória. É justamente nesses pontos que a análise dinâmica se torna necessária. Ao mesmo tempo, a análise dinâmica também possui limites, pois depende de ambiente, dados e cenários realistas de teste. Mesmo após o uso das ferramentas, ainda podem permanecer dificuldades como regras de negócio escondidas no código, integrações pouco documentadas e dependências externas críticas.

6. Conclusão

A principal aprendizagem deste trabalho é que a reengenharia de software depende de compreensão progressiva e bem fundamentada. Em sistemas legados, recuperar conhecimento é a etapa que torna possíveis intervenções mais seguras. Entre as ferramentas estudadas, a que mais se destacou para iniciar o processo foi o SonarQube, porque oferece uma visão ampla da qualidade estrutural do sistema e apoia a priorização técnica.

Ainda assim, a recomendação final não deve ser baseada em uma única ferramenta. Para o cenário do enunciado, a melhor estratégia inicial é usar o SonarQube para mapear criticidade, aplicar o Checkstyle para reduzir inconsistências de codificação e recorrer ao VisualVM quando for preciso confirmar sintomas de execução. Essa combinação ajuda a diminuir risco, custo e esforço de manutenção, criando uma base mais confiável para as próximas etapas da reengenharia.